

AhamX Bodhi

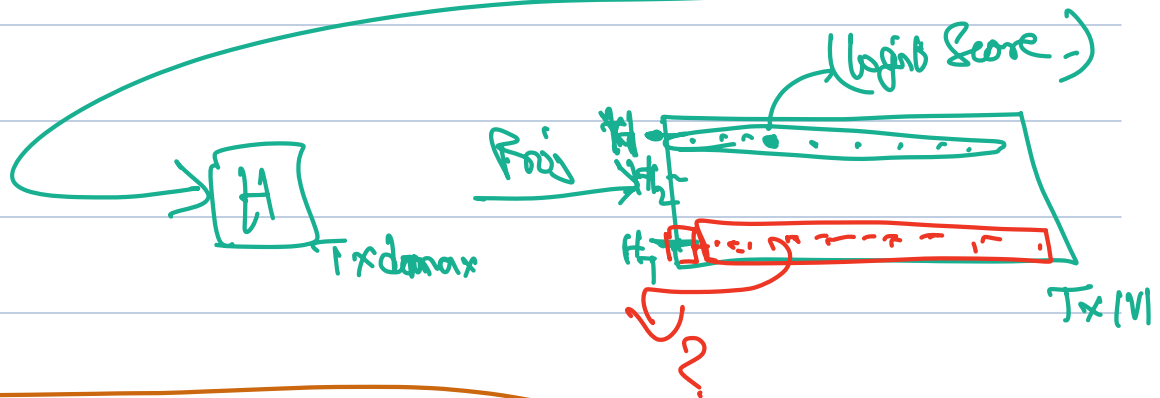
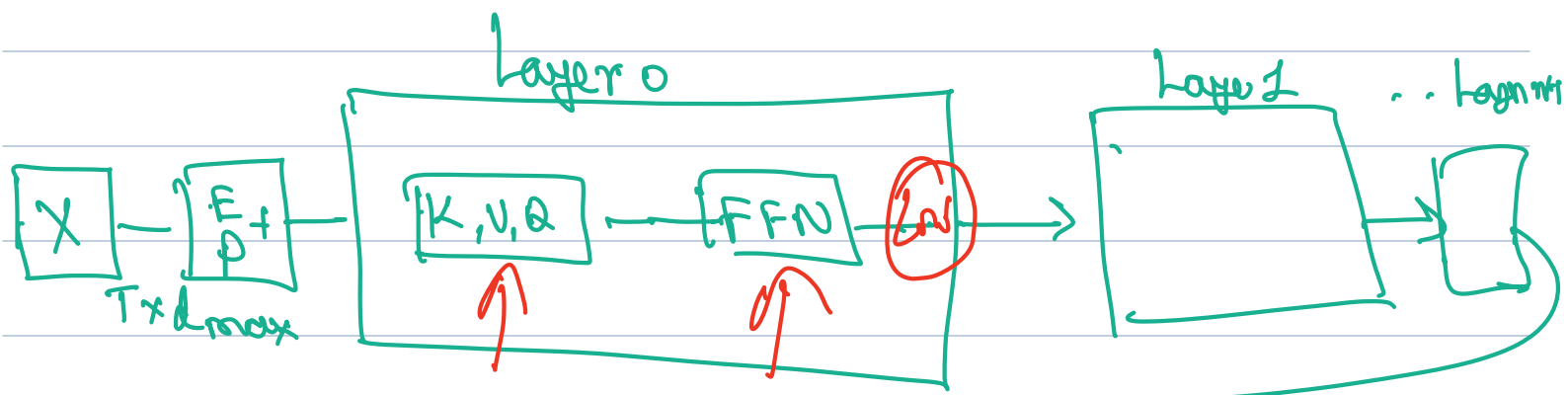
Where I Becomes Infinite

API Fundamentals for LLMs

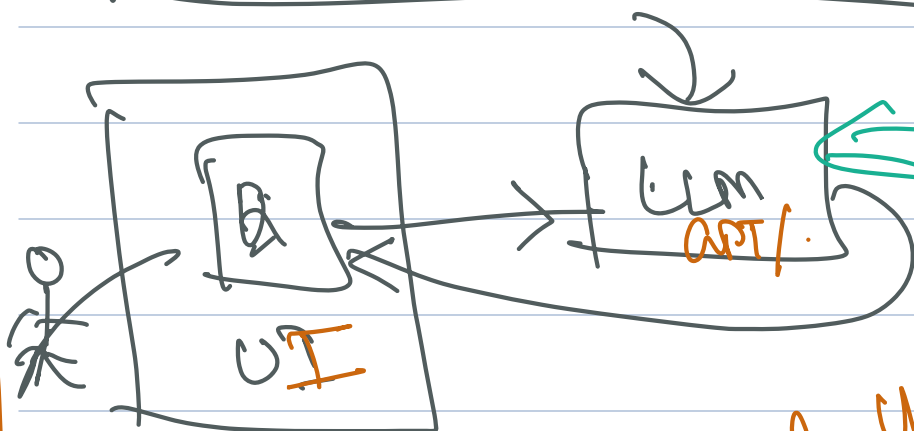
Module 3.1: Working with LLM APIs

Prof. Sashikumaar Ganesan | IISc Bengaluru | 2026





Chat GPT / Gemini / Claude



General users

API

User own Interface / Code

→ Agentic



Session Overview

What You Will Learn

- What an LLM API is and why it exists
- The request-response cycle in detail
- Core API parameters and their purpose
- Authentication, rate limits, and cost
- How major providers structure their APIs

Concept at a Glance

- **Duration:** 8 minutes
- **Bloom's Level:** B2 – Understand
- **Difficulty:** L2 – Intermediate
- **Module:** 3.1 Working with LLM APIs
- **Prerequisite:** What is a Language Model

Learning Objectives



By the End of This Concept You Will Be Able To

- **Define** what an LLM API is and explain the role it plays between an application and a hosted language model
- **Describe** the structure of an API request: endpoint, headers, and JSON body with model, messages, and parameters
- **Identify** the core parameters — model, messages, temperature, max_tokens — and state the function of each
- **Explain** authentication via API keys and the implications for security and cost management
- **Compare** the APIs of OpenAI, Anthropic, and Google at a structural level



What is an LLM API?

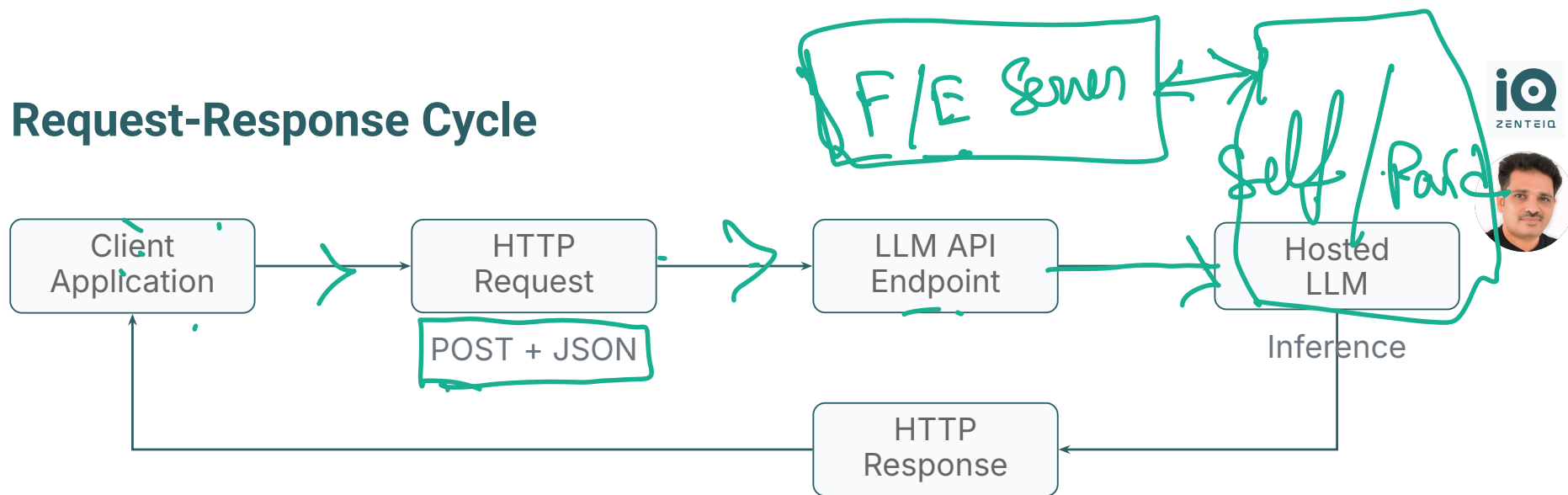
Application Programming Interface for Language Models

An **LLM API** is a web service that exposes the capabilities of a hosted language model over HTTP. A client application sends a structured request (prompt + parameters) and receives a structured response (generated text + metadata) — without needing to know anything about the underlying model weights or infrastructure.

Why APIs Instead of Running Models Locally?

- Large models (GPT-4, Claude 3) require hundreds of GBs of GPU memory – impractical for most developers to host
- API providers handle hardware, scaling, and model updates
- Pay-per-token pricing aligns cost with usage
- Access to the latest model versions without local re-deployment

The Request-Response Cycle



Each API Call Is a Stateless HTTP Transaction

The client sends a **POST** request to the provider endpoint. The model runs a full forward pass and returns the generated tokens as JSON. The API server holds no memory between calls — conversation history must be sent explicitly in every request.

Anatomy of an API Request



Private / Paid
LLM service

Three Required Components of Every LLM API Call

- **Endpoint URL:** The HTTPS address of the API service. Example: `https://api.openai.com/v1/chat/completions`
- **Headers:** Contain the Authorization: Bearer <API_KEY> token and Content-Type: `application/json`
- **Body (JSON):** Contains `model` (which LLM to use), `messages` (conversation history as a list of role/content objects), and optional sampling parameters such as `temperature` and `max_tokens`

The Messages Array is the Core Abstraction

Every API call wraps the conversation as an ordered list of messages, each with a **role** (system, user, or assistant) and **content** (text). This single structure handles single-turn Q&A, multi-turn chat, and system-instructed interactions uniformly.



The Messages Object --- Roles and Content

Three Message Roles

- **system**: Sets the model's persona and constraints. Processed first; acts as standing instructions throughout the conversation
- **user**: Represents the human's turn — queries, instructions, or content to process
- **assistant**: Represents prior model responses. Included to give the model conversation history

Why Roles Matter

- The model is trained to respond differently based on role
- System prompt content is typically more authoritative than user content due to RLHF training
- Including prior assistant turns enables coherent multi-turn dialogue
- Omitting history makes every call completely independent (stateless by design)





Core API Parameters at a Glance

Parameter	Type	Effect
model	string	Selects which LLM to invoke (e.g., gpt-4o)
messages	array	Full conversation history (role + content)
temperature	float [0, 2]	Controls output randomness and creativity
max_tokens	integer	Maximum tokens in the generated response
top_p	float [0, 1]	Nucleus sampling cutoff probability
top_k	integer	Limits candidate tokens at each step
stream	boolean	Returns tokens incrementally as SSE events
stop	string/list	Stop generation at specified string(s)



Authentication --- API Keys

API Keys: Critical Security Considerations

- An API key is a secret credential that authorises all charges to your account — treat it like a password
- **Never** hardcode keys in source code or commit them to version control (GitHub secret scanning will flag this)
- Store keys in environment variables (.env files) or secret management services (AWS Secrets Manager, HashiCorp Vault)
- Rotate keys immediately if exposed; set spending limits as a budget safety net

Standard Header Format

Every request includes the header:

`Authorization: Bearer sk-...your-key-here...`

The provider validates the key, identifies the account, and applies rate limits and billing against that account for the request.



Rate Limits and Token Quotas

What Rate Limits Control

- **RPM:** Requests per minute limit
- **TPM:** Tokens per minute (input + output)
- **TPD:** Tokens per day (daily cap)
- Limits increase with account tier and usage history
- Hitting a limit returns HTTP 429 (Too Many Requests)

Handling Rate Limits in Code

- Implement exponential backoff with jitter on 429 responses
- Cache responses for repeated identical prompts
- Use batching for bulk, non-latency-critical workloads
- Monitor token usage via provider dashboards
- Consider smaller models for high-volume use cases



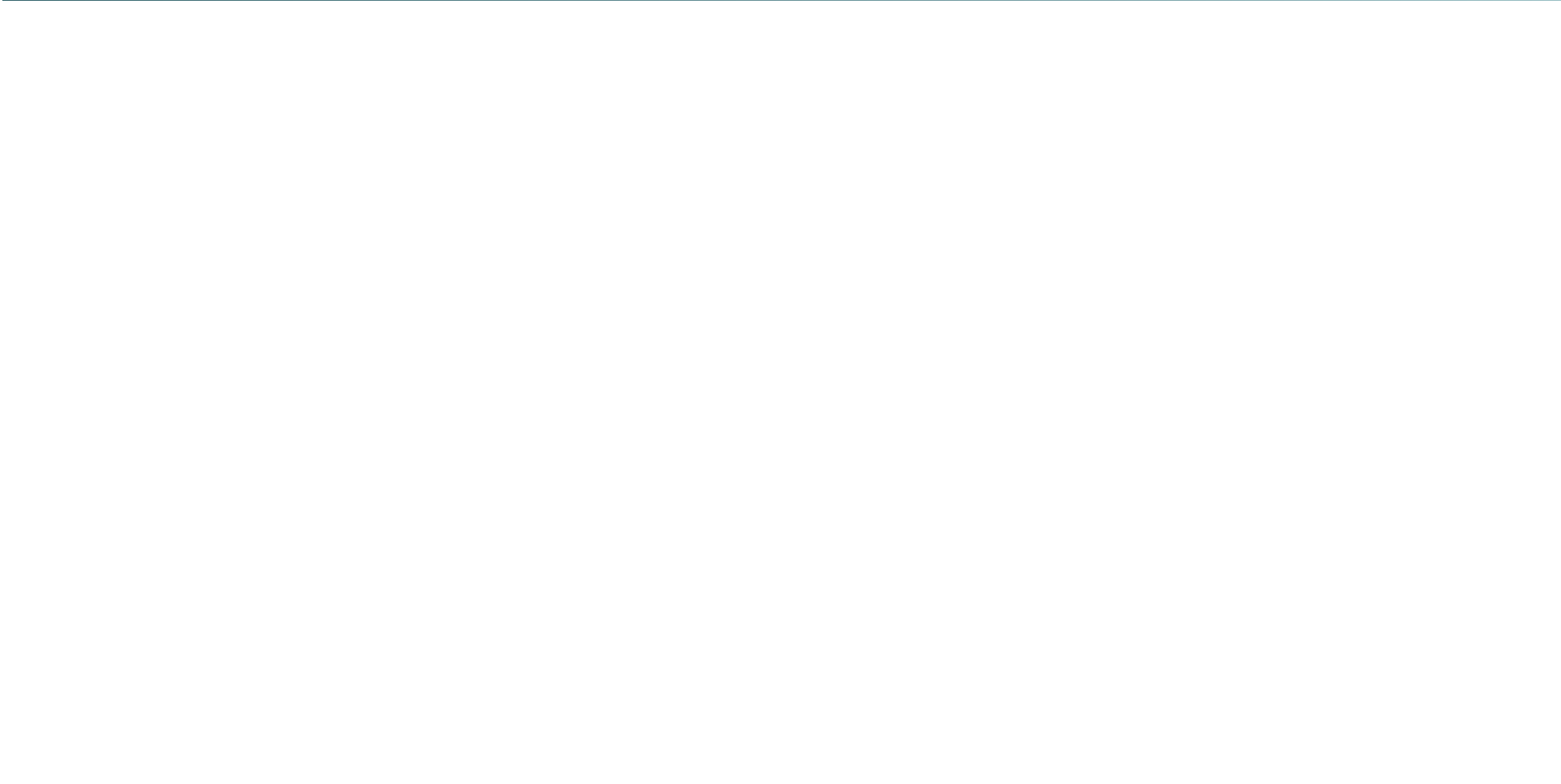
Understanding Token-Based Pricing

How LLM API Costs Work

Providers charge per 1,000 or 1,000,000 tokens processed. Cost has two components: **input tokens** (your prompt + conversation history) and **output tokens** (the model's response). Output tokens are typically 2–3× more expensive than input tokens due to the sequential, non-parallelisable nature of generation.

Cost Implications for Developers

- Long system prompts are paid for on *every* API call
- Multi-turn conversations grow in cost as history accumulates
- `max_tokens` caps output cost but does not affect input cost
- Prompt caching (Anthropic, OpenAI) can reduce repeated prefix costs by up to 90%
- Choosing a smaller model (GPT-4o-mini vs GPT-4o) can reduce cost by 10–20× for suitable tasks





OpenAI API --- The Industry Standard

OpenAI Chat Completions API

The **OpenAI API** at `api.openai.com/v1/chat/completions` is the most widely adopted LLM API and has become a de facto standard. Many open-source serving frameworks (vLLM, Ollama, LM Studio) implement an OpenAI-compatible API so existing client code works without modification.

Key Features of the OpenAI API

- **Models:** GPT-4o, GPT-4o-mini, GPT-4-Turbo, o1, o3
- **Function Calling:** Structured tool-use with JSON schema
- **Streaming:** Server-sent events for real-time token delivery
- **Vision:** Multimodal inputs (images + text) in the same request
- **Assistants API:** Stateful threads with file retrieval and code interpreter tools



Anthropic API --- Safety-Focused Design

Anthropic Messages API

The **Anthropic API** at `api.anthropic.com/v1/messages` provides access to Claude models. While structurally similar to the OpenAI API, it separates `system` as a top-level parameter (not part of the messages array) and uses `anthropic-version` header for versioning.

Key Features of the Anthropic API

- **Models:** Claude 3.5 Sonnet, Claude 3.5 Haiku, Claude 3 Opus
- **Context Window:** Up to 200K tokens — largest among major providers
- **Prompt Caching:** Cache static prefixes to reduce cost and latency
- **Tool Use:** Structured function calling with XML-based tool definitions
- **Constitutional AI:** Built-in safety training reduces refusal engineering



Google Gemini API --- Multimodal Native

Google Generative AI API

The **Google Gemini API** via Vertex AI and `generativelanguage.googleapis.com` provides access to Gemini 1.5 Pro, Gemini 1.5 Flash, and Gemini 2.0 models. Gemini was designed as multimodal from the ground up, handling text, images, audio, and video in a single API call.

Key Features of the Google Gemini API

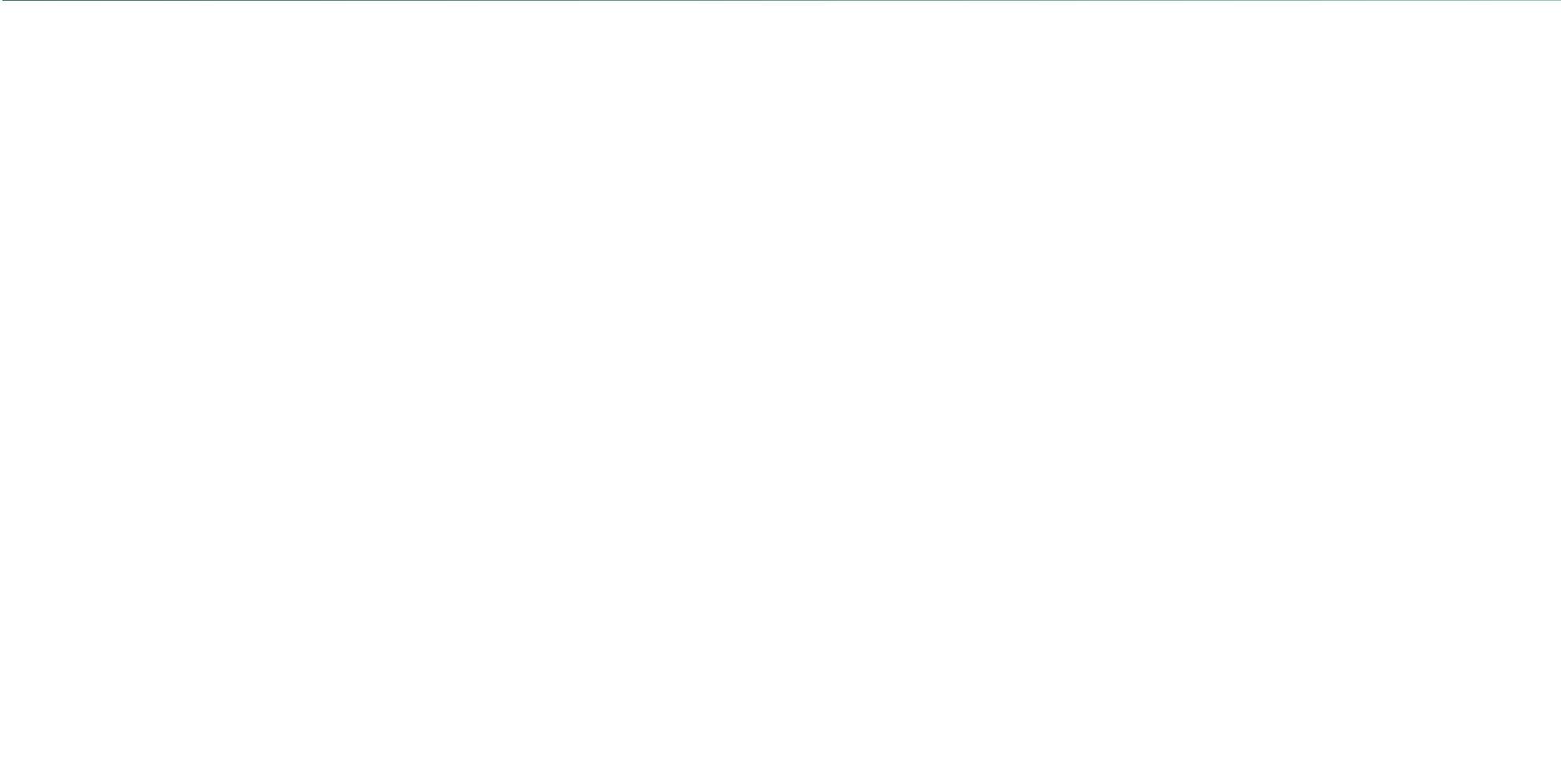
- **Context Window:** 1M tokens for Gemini 1.5 Pro — enables full-document and video analysis
- **Native Multimodal:** Audio, video, images, and text in one request
- **Grounding:** Built-in Google Search grounding reduces hallucinations
- **Free Tier:** Generous free quota for development and prototyping
- **Vertex AI:** Enterprise deployment with VPC isolation and audit logs

LLM API Comparison --- Major Providers



Feature	OpenAI	Anthropic	Google Gemini
Base URL	api.openai.com	api.anthropic.com	generativelanguage.googleapis.com
Top Model	GPT-4o	Claude 3.5 Sonnet	Gemini 1.5 Pro
Max Context	128K tokens	200K tokens	1M tokens
System Prompt	In messages array	Top-level field	systemInstruction
Auth Header	Bearer token	x-api-key	API key / OAuth2
Streaming	SSE	SSE	SSE
Tool/Function Use	Yes	Yes	Yes
Prompt Caching	Yes	Yes	Yes (Vertex)

All three APIs accept JSON bodies and return JSON responses. SDK libraries (Python, Node.js) abstract the HTTP layer for all providers.





Streaming Responses --- Real-Time Token Delivery

Without Streaming

- The API waits until generation is complete
- Single JSON response returned at once
- User sees nothing until the full response arrives
- Higher perceived latency, especially for long outputs
- Simpler to implement: just parse one JSON object

With Streaming (`stream: true`)

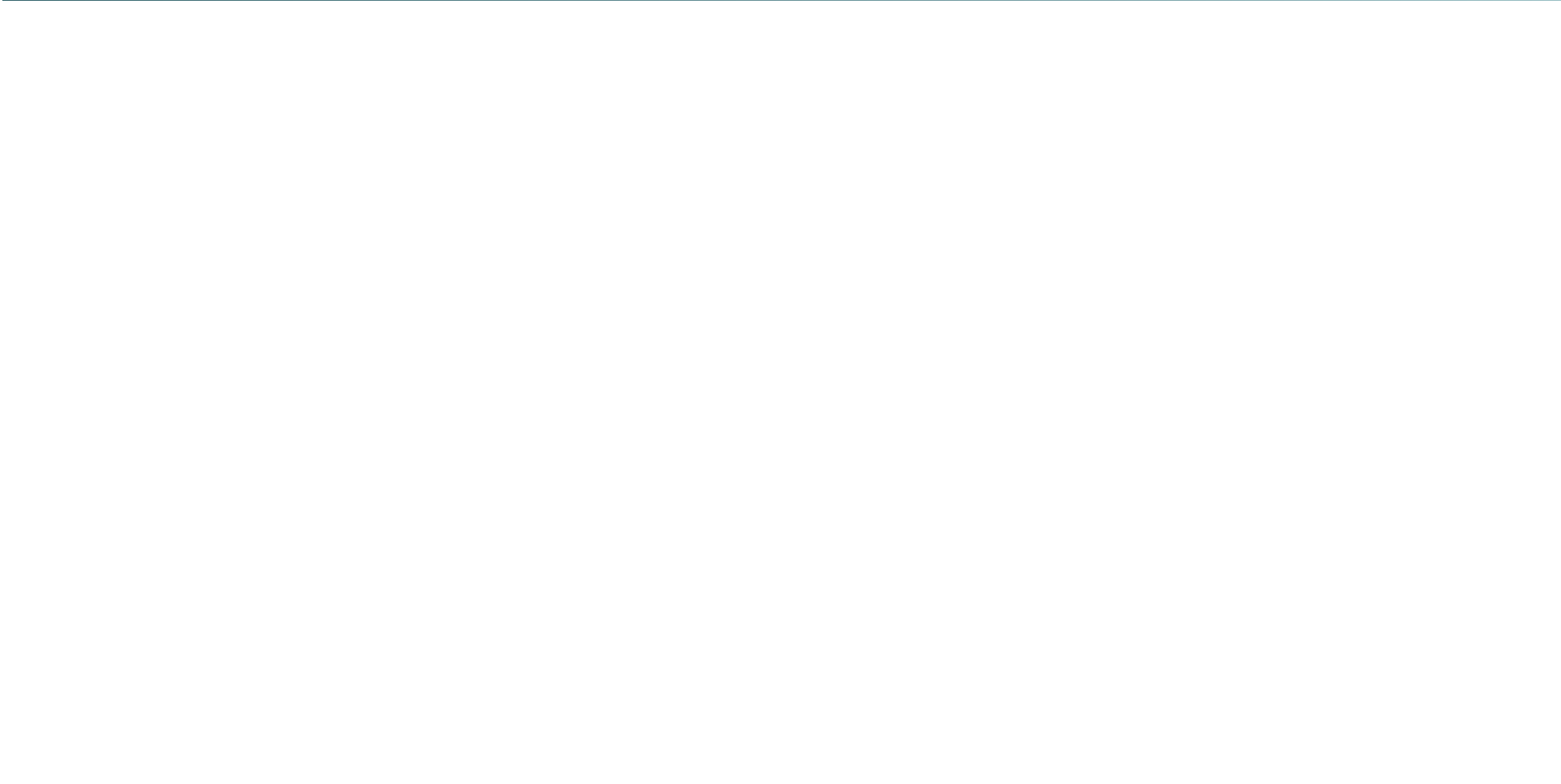
- Server-Sent Events (SSE) stream token chunks as generated
- First token arrives within hundreds of milliseconds
- User sees text appearing progressively (ChatGPT UX)
- Lower perceived latency; better for interactive applications
- Requires handling partial JSON chunks in client code



HTTP Status Codes in LLM APIs

Code	Name	Meaning and Action
200	OK	Success — parse the response JSON
400	Bad Request	Malformed JSON or invalid parameter — fix client code
401	Unauthorized	Missing or invalid API key — check credentials
403	Forbidden	Key valid but lacks permission for this model
429	Too Many Requests	Rate limit exceeded — backoff and retry
500	Server Error	Provider-side issue — retry with exponential backoff
503	Service Unavailable	Provider overloaded — retry after delay

Robust LLM applications handle 429 and 5xx errors with automatic retry logic. The `tenacity` library in Python provides production-ready retry decorators.





GenAI Application: Conversational AI Products

How ChatGPT, Claude.ai, and Gemini Use Their Own APIs

- **ChatGPT (OpenAI):** The web interface sends every user message as a `/v1/chat/completions` request with the full conversation history. The UI streams back tokens via SSE
- **Claude.ai (Anthropic):** Uses the Messages API with a persistent system prompt defining Claude's behavior, and streams the assistant turn
- **Gemini (Google):** Combines API calls with real-time Search grounding to augment responses with up-to-date web results
- **Microsoft Copilot:** Wraps OpenAI's API with Bing search retrieval, injecting web context into the prompt before each call

The consumer products you use daily are themselves API clients — understanding the API gives you insight into exactly how these systems work.



GenAI Application: GitHub Copilot Code Completion

Real-Time Code Generation via LLM API

- **Trigger:** As you type, the IDE extension sends your current file and cursor context to the OpenAI Codex / GPT-4 API
- **System prompt:** Contains instructions about code style, language, and completion behavior
- **User message:** The current file content (truncated to fit context window) with a special token marking the cursor position
- **Parameters:** Low temperature (≈ 0.1) for deterministic, syntactically correct completions; limited `max_tokens` for fast response
- **Streaming:** Tokens stream back and appear inline as ghost text before the developer accepts the suggestion

Every keystroke delay you see in Copilot is API latency — network + model inference.



GenAI Application: Retrieval-Augmented Generation

LLM API as the Reasoning Layer in RAG

- **Step 1:** User query is embedded and sent to a vector database (Pinecone, Weaviate) to retrieve relevant document chunks
- **Step 2:** Retrieved chunks are concatenated with the user query and injected into the user or system message
- **Step 3:** The enriched prompt is sent to the LLM API (typically GPT-4o or Claude 3.5 Sonnet) for synthesis
- **Step 4:** The LLM generates an answer grounded in the retrieved context, reducing hallucinations compared to prompting without context
- **Cost implication:** Long retrieved contexts dramatically increase input token cost — chunk size optimization is critical

RAG pipelines (LangChain, LlamaIndex) automate Steps 1–4 with a few lines of code.



GenAI Application: AI Agents and Tool Calling

The Agent Loop via API

- Agent sends prompt + tool definitions to LLM API
- Model returns a `tool_call` JSON object
- Agent executes the tool (web search, code, DB query)
- Tool result is appended to messages as a `tool` role
- API is called again with updated history
- Loop continues until model returns plain text (final answer)

Production Agent Frameworks

- **LangChain Agents:** Wraps the API loop with tool registry and memory management
- **OpenAI Assistants API:** Manages thread state server-side, simplifying agent persistence
- **Anthropic Claude Agents:** Computer use API enables agents to interact with real GUIs
- **AutoGPT / CrewAI:** Multi-agent frameworks orchestrating multiple API clients



Common Pitfalls in LLM API Integration

Mistakes to Avoid in Production

- **No retry logic:** A single 429 or 500 error crashes the application — always implement exponential backoff
- **Hardcoded API keys:** Exposes credentials in version control; use environment variables exclusively
- **Ignoring token limits:** Sending history that exceeds the context window silently truncates early messages
- **No timeout:** LLM calls can take 30–60 seconds for long outputs — set explicit HTTP timeouts to prevent hanging
- **Parsing the wrong field:** Response structure differs between providers; use the official SDK to avoid field-name errors

Knowledge Connections



What You Already Know

- What a Language Model is and how it generates text
- Autoregressive token-by-token generation
- Temperature, top-p, and sampling strategies
- Tokenization and context windows

What This Unlocks

- Parameter tuning in API calls (Temperature, Top-P, Top-K)
- Building RAG pipelines with LangChain
- Prompt engineering with system roles
- Fine-tuning via OpenAI fine-tuning API
- Agentic workflows with tool calling

Session Summary



Key Takeaways

1. An **LLM API** is a stateless HTTP service: send a JSON body with model + messages + parameters, receive generated text
2. The **messages array** with system/user/assistant roles is the universal abstraction for conversation state
3. **API keys** authenticate requests and determine billing — never expose them in code or logs
4. **Rate limits** (RPM, TPM) require backoff logic; token pricing rewards concise prompts
5. **OpenAI, Anthropic, and Google** expose structurally similar APIs differing mainly in URL, auth headers, and model names
6. **Streaming** enables low-latency UX; **tool calling** enables agent loops; both build on the same core API

AhamX Bodhi

Where I Becomes Infinite